

Compiled Knowledge as an Intermediate Representation for AI Agents:

A Longitudinal Evaluation of Semantic Preservation, Section Allocation, and Package Sanitation in the CKF Compiler

Paulo Tomazinho, PhD
CKF Research
paulo@tomazinho.com.br

May 2026

Abstract

AI agents increasingly consume documents that were designed for human readers rather than machine reasoning. This creates a representational mismatch: narrative documents encode knowledge through prose, sequence, headings, implicit scope, and rhetorical contrast, whereas agents operate over retrieved spans, schemas, tool calls, context windows, and provenance constraints. This paper presents the Compiled Knowledge Format (CKF) as an intermediate representation for knowledge: a structured, source-grounded package format that compiles human documents into machine-operable units for retrieval systems and agents. We report a longitudinal artifact-and-compiler study of CKF across six stages: a shallow CKF-0.1-style baseline, compiler v1.0, v1.01, v1.02, v1.03, and v1.03.1. Using a stable reference document, *What CKF is not (and what it actually is)*, and a 20-unit semantic preservation rubric, we evaluate each stage by semantic preservation, structural population, section allocation, retrieval readiness, source traceability, language fidelity, metadata calibration, and package sanitation behavior. The shallow baseline preserved roughly 10% of the semantic rubric. Compiler v1.0 raised preservation to 97.5% but concentrated knowledge in atomic units and retrieval chunks. Compiler v1.01 achieved 100% preservation by promoting extracted knowledge into richer normative and operational sections. Compiler v1.02 retained full preservation while adding calibrated metadata, section-allocation scoring, and partial if-then/playbook promotion. Compiler v1.03 introduced global package sanitation and language locking, removing visible language drift and truncation artifacts, but over-sanitized the package by deleting all retrieval chunks, procedures, if-then rules, and playbooks. Compiler v1.03.1 corrected this regression with a field-aware sanitizer, retrieval-chunk preservation by body validity, and source-traceability rebuilding. The final v1.03.1 package preserved 100% of the semantic rubric, contained 511 counted CKF elements, preserved 18 retrieval chunks, 66 atomic units, 4 procedures, 3 playbooks, 36 question-answer pairs, and 261 traceability entries, and achieved calibrated metadata scores of human-readability 0.8, AI-utility 1.0, and section-allocation quality 1.0. The study shows that reliable knowledge compilation is multi-dimensional: semantic preservation, section allocation, retrieval readiness, language fidelity, traceability consistency, and sanitation precision can improve or regress independently. We conclude that CKF compilation should be evaluated as a compiler problem rather than as a serialization problem, and we propose design principles for preservation-oriented, field-aware knowledge compilers.

Keywords: Compiled Knowledge Format; CKF; knowledge representation; AI agents; semantic preservation; retrieval-augmented generation; GraphRAG; Model Context Protocol; provenance; package sanitation; section allocation; language fidelity; intermediate representation; compiler architecture

1. Introduction

Large language model (LLM) systems increasingly operate over documents that were designed for humans rather than machines. PDF pages, policy manuals, standards documents, scientific articles, web pages,

and prose-heavy Markdown files encode knowledge through narrative sequence, headings, emphasis, implicit scope, exception structure, and human interpretive convention. Agents, by contrast, operate through context windows, retrieved chunks, function calls, structured inputs, schemas, and tool-mediated state. This mismatch raises a practical question for knowledge engineering: when a human-readable document is converted into a machine-readable artifact, how much of the source’s useful knowledge survives, and in what form?

The Compiled Knowledge Format (CKF) addresses this layer of the AI stack. CKF is not a vector database, graph database, ontology, fine-tuning method, RAG architecture, GraphRAG system, Model Context Protocol (MCP), PDF replacement, or prompt library. CKF is a structured content format: a compiled intermediate representation for knowledge. It is designed to sit between human documents and runtime systems such as retrieval pipelines, vector databases, graph stores, MCP resources, and AI agents.

The central hypothesis is that structural inference should move from inference time to compile time. In a conventional retrieval pipeline, a model retrieves unstructured text chunks and must reconstruct scope, hierarchy, exception, relation, and applicability at answer time. In a CKF-aware pipeline, a compiler has already converted the source into typed, source-grounded units: entities, concepts, principles, decision rules, procedures, anti-patterns, exceptions, playbooks, question-answer pairs, retrieval chunks, atomic units, and traceability records.

This paper reports a longitudinal study of CKF compiler evolution from a shallow baseline to compiler deployment v1.03.1. Unlike a single final-result paper, this study treats compiler development as an empirical sequence. Each version addressed a specific failure mode and sometimes introduced a new one. The resulting arc is scientifically useful because it shows that knowledge compilation quality is not one-dimensional. A compiler may improve semantic preservation while failing section allocation; improve sanitation while damaging retrieval readiness; or improve language fidelity while reducing operational usefulness.

The final v1.03.1 compiler is the first version in this study to jointly satisfy four major requirements: full semantic preservation, strong section allocation, global language/truncation sanitation, and preserved retrieval surface. This paper therefore frames v1.03.1 as the current best evidence for CKF’s feasibility as a preservation-oriented intermediate representation for AI-consumable knowledge.

2. Contributions

This paper makes five contributions:

1. It formalizes CKF as a compiled intermediate representation for knowledge, distinct from databases, graphs, ontologies, protocols, prompts, and model weights.
2. It presents a 20-unit semantic preservation rubric for evaluating whether CKF compilation preserves substantive source knowledge.
3. It reports a longitudinal comparison across CKF-0.1-style, v1.0, v1.01, v1.02, v1.03, and v1.03.1 compiler outputs.
4. It identifies section allocation, package sanitation, language fidelity, traceability consistency, and retrieval readiness as distinct quality dimensions for knowledge compilers.
5. It derives compiler design principles from observed successes and regressions, culminating in a field-aware sanitizer that preserves valid retrieval chunks while removing language drift and truncation artifacts.

3. Background

3.1. Retrieval-Augmented Generation and Structured Retrieval

Retrieval-Augmented Generation (RAG) retrieves external content at query time and passes it to a generative model [8]. Subsequent work has explored reflective and adaptive retrieval, including Self-RAG [1]. GraphRAG extends retrieval with graph construction and global summarization over entity networks [4]. These systems address how to retrieve and reason over information. CKF addresses what the retrievable information looks like before retrieval begins.

In a conventional RAG pipeline, the retrieval substrate is usually an unstructured chunk. In a CKF-aware pipeline, the retrieval substrate can be a typed unit with provenance and an operational role. A vector database can index CKF atomic units or retrieval chunks. A graph store can ingest CKF entities and relations. An MCP resource can serve a CKF package as structured content. The infrastructure remains; the representation changes.

3.2. Hallucination, Attribution, and Provenance

Hallucination research distinguishes unsupported generation, factual inconsistency, and grounding failure [5]. CKF is especially concerned with composition hallucination: failure to reconstruct relations among fragments under retrieval pressure. Attribution frameworks such as AIS evaluate whether generated statements are supported by identified sources [11]. The W3C PROV model treats provenance as representable information about entities, activities, and agents [12, 10]. CKF operationalizes this idea by embedding source references, source excerpts, source basis, and confidence into extracted items.

3.3. Serialization Is Not Compilation

JSON, YAML, Markdown, and PDF serve different purposes. PDF preserves presentation for human readers [6]. JSON supports data interchange [2]; JSON Schema supports validation [7]. Markdown supports lightweight human-readable technical text [3]. YAML supports hierarchical configuration and data serialization [13]. CKF can be serialized as JSON, Markdown, or YAML, but CKF quality is not determined by serialization alone. The relevant question is whether the compiler preserves and allocates the source’s knowledge.

4. Research Questions

The study asks:

RQ1. How did semantic preservation evolve across CKF compiler versions?

RQ2. How did structural population and section allocation change from v1.0 to v1.03.1?

RQ3. Which compiler mechanisms explain the improvements and regressions?

RQ4. What did v1.03 reveal about the risks of global package sanitation?

RQ5. Did v1.03.1 resolve the v1.03 over-sanitization regression?

RQ6. What remaining limitations should guide the next compiler iteration?

5. Materials and Methods

5.1. Reference Document

The evaluation uses the reference document *What CKF is not (and what it actually is)*. The document is short but semantically dense. It defines CKF by contrast with vector databases, knowledge graphs,

ontologies, fine-tuning, RAG, GraphRAG, MCP, PDF, and prompt libraries. It also defines CKF as an open structured knowledge format, introduces operational fidelity, compares CKF to LLVM IR, and states an empirical research program around retrieval precision, faithfulness, token efficiency, auditability, and composition hallucinations.

5.2. Compiler Stages

We compare six stages:

1. a shallow CKF-0.1-style baseline;
2. CKF compiler v1.0;
3. CKF compiler v1.01;
4. CKF compiler v1.02;
5. CKF compiler v1.03;
6. CKF compiler v1.03.1.

The deployment version changes, but the serialized protocol for the later artifacts remains `ckf-1.0`. This distinction is important: compiler evolution does not necessarily imply schema incompatibility.

5.3. Semantic Preservation Rubric

The source document was decomposed into 20 substantive knowledge units. Each CKF artifact received 1.0 if the unit was clearly preserved, 0.5 if partially preserved, and 0.0 if absent.

The preservation score is:

$$\text{Preservation score} = \frac{\sum_{i=1}^{20} s_i}{20}. \quad (1)$$

5.4. Structural Population and Artifact Metrics

For each compiler stage we counted CKF section populations: entities, concepts, principles, heuristics, decision rules, procedures, patterns, anti-patterns, causal chains, contextual triggers, if-then rules, exceptions, mental models, playbooks, question-answer pairs, retrieval chunks, atomic units, and source traceability.

For v1.03.1 we measured bytes, kilobytes, page count where applicable, line count, character count, and word count across PDF, JSON, Markdown, and YAML artifacts. We also audited compiler source files: semantic chunker, structured extraction schema, reducer, promotion module, text filters, quality gates, global package sanitizer, compiler pipeline, provider adapters, and regression tests.

6. Results

6.1. v1.03.1 Artifact Metrics

Table 1 reports physical metrics for the v1.03.1 artifacts.

The CKF artifacts are larger than the v1.03 artifacts because v1.03.1 restores retrieval chunks, atomic units, procedures, playbooks, and other structures that v1.03 had over-sanitized.

Artifact	Ext.	Bytes	KB	Pages	Lines	Words	Role
PDF source	.pdf	304,392	297.26	7	240	2,067	Human-readable source
JSON CKF v1.03.1	.json	209,128	204.23	–	5,602	19,966	Programmatic package
Markdown CKF v1.03.1	.md	172,118	168.08	–	3,925	19,734	Human-auditable package
YAML CKF v1.03.1	.yaml	174,653	170.56	–	3,923	19,966	Compact structured package

Table 1: Physical metrics for the CKF v1.03.1 artifacts.

Section	JSON	Markdown	YAML
entities	12	12	12
concepts	17	17	17
principles	46	46	46
heuristics	0	0	0
decision_rules	6	6	6
procedures	4	4	4
patterns	0	0	0
anti_patterns	25	25	25
causal_chains	0	0	0
contextual_triggers	0	0	0
if_then_rules	0	0	0
exceptions	15	15	15
mental_models	2	2	2
playbooks	3	3	3
qa_pairs	36	36	36
retrieval_chunks	18	18	18
atomic_units	66	66	66
source_traceability	261	261	261
Total counted elements	511	511	511

Table 2: Structural population of the v1.03.1 CKF artifacts.

6.2. v1.03.1 Structural Population

Table 2 shows the section population for v1.03.1.

The final v1.03.1 package restores the retrieval surface that v1.03 had removed. It contains 18 retrieval chunks and 66 atomic units, plus 4 procedures and 3 playbooks. This establishes v1.03.1 as a recovery from v1.03’s over-sanitization regression.

6.3. Structural Density

Table 3 shows structural density.

Format	Counted elements	KB	Elements/KB
JSON	511	204.23	2.50
Markdown	511	168.08	3.04
YAML	511	170.56	3.00

Table 3: Structural density for v1.03.1.

Markdown has the best density in this run while remaining suitable for human audit. JSON remains the most appropriate format for strict programmatic consumption.

6.4. Semantic Preservation

Table 4 reports v1.03.1 semantic preservation.

Table 4: Semantic preservation rubric for CKF v1.03.1.

#	Knowledge unit	Score
1	CKF is often confused with technologies that it complements.	1
2	CKF is not a vector database.	1
3	CKF composes with vector databases: the database indexes CKF units.	1
4	CKF is not a knowledge graph or graph database.	1
5	CKF is broader than a graph: it includes rules, exceptions, procedures, playbooks, and mental models.	1
6	CKF composes with graph stores.	1
7	CKF is not an ontology; it is local, automatically compiled, and versioned with its source.	1
8	CKF is not fine-tuning; it is data, not model weights.	1
9	CKF packages can be updated, audited, versioned, and replaced without retraining.	1
10	CKF does not replace RAG; RAG is an architecture and CKF is a content format.	1
11	CKF changes one box in the RAG pipeline: unstructured chunks become typed CKF units.	1
12	The CKF hypothesis concerns gains in faithfulness, precision, and resistance to composition hallucinations under retrieval pressure.	1
13	CKF does not replace GraphRAG; GraphRAG focuses on connections while CKF focuses on operation, rules, and exceptions.	1
14	CKF does not replace MCP; MCP is the protocol layer and CKF is a possible knowledge payload format.	1
15	PDF preserves visual fidelity for humans; CKF preserves operational fidelity for machine readers.	1
16	CKF moves structural inference from inference time to compile time.	1
17	CKF is not a prompt library; it separates knowledge from agent behavior.	1
18	CKF is an open file format for structured knowledge consumed by agents.	1
19	A CKF package contains entities, concepts, rules, exceptions, procedures, principles, heuristics, causal chains, atomic units, and provenance.	1
20	CKF functions as a compiled intermediate representation analogous to LLVM IR and must demonstrate adoption empirically, including the pre-registered Study 2.	1
Total		20/20
Preservation score		100%

The v1.03.1 artifacts preserve the full rubric. This restores the 100% preservation observed in v1.01 and v1.02, after v1.03 had dropped to 97.5% due to partial loss of the explicit Study 2 reference.

6.5. Longitudinal Comparison

Table 5 compares the compiler stages.

The table shows why v1.03.1 is the best current compiler deployment. v1.03 solved language and truncation issues but broke retrieval readiness. v1.03.1 preserves those fixes while restoring retrieval chunks, atomic units, procedures, and playbooks.

Criterion	v1.0	v1.01	v1.02	v1.03	v1.03.1
Semantic preservation	97.5%	100%	100%	97.5%	100%
Total counted	224	388	393	393	511
Entities	15	13	15	17	12
Concepts	14	16	16	17	17
Principles	0	25	20	42	46
Decision rules	0	9	8	4	6
Procedures	0	4	5	0	4
Anti-patterns	0	8	11	4	25
Exceptions	0	5	5	15	15
If-then rules	1	0	2	0	0
Playbooks	0	0	3	0	3
QA pairs	0	27	27	37	36
Retrieval chunks	23	21	23	0	18
Atomic units	56	64	61	39	66
Source traceability	112	194	194	256	261
Language drift	Present	Present	Partial	Corrected	Corrected
Truncation artifacts	Present	Present	Partial	Corrected	Corrected
Over-sanitization	No	No	No	Yes	Corrected

Table 5: Longitudinal comparison of CKF compiler stages.

7. Compiler Architecture Audit

7.1. Semantic Chunking

The chunker detects Markdown headings, chapters, numbered sections, decorative separators, and uppercase title lines. Each chunk receives a stable span identifier such as `s_001`. This supports source-level traceability and allows extracted items to cite the source span from which they were derived.

7.2. Traceable Structured Extraction

The extraction schema requires item-level source basis, confidence, source references, and source excerpts. This transforms provenance from a post-hoc annotation into a core property of the representation.

7.3. Reduction and Promotion

The reducer merges per-chunk partial packages into a canonical package and promotes selected atomic units and retrieval chunks into richer sections. Promotion is conservative: it duplicates eligible material rather than removing atomic or retrieval units. This was the key mechanism behind the v1.01 improvement in section allocation.

7.4. Quality Gates

The quality module computes coverage, density, traceability, schema completeness, section allocation quality, human readability, AI utility, language fidelity, and promotion acceptance. These scores make compiler quality observable, although they should be interpreted diagnostically rather than as complete proof of correctness.

7.5. Language Lock and Short-String Detection

The compiler includes a language lock and short-string language signals for English/Portuguese detection. This resolves the earlier failure in which strings such as “Camada Operacional” and “Fidelidade Visual” could survive in an English package because short strings were not reliably detected by statistical language identification.

7.6. Global Package Sanitizer

Compiler v1.03 introduced a global package sanitizer, but applied proposition-completeness checks too broadly. Compiler v1.03.1 corrected this by making the sanitizer field-aware. Fields are assigned roles such as label, title, name, tag, substantive, and source excerpt. Completeness checks apply only to substantive fields, while labels and titles receive only language and obvious-truncation checks. Source excerpts remain verbatim and are exempt from language mismatch filtering.

7.7. Retrieval Chunk Preservation

v1.03.1 adds a retrieval-specific preservation rule. A retrieval chunk should not be removed merely because its title is short or title-like. If the body - especially `compressed_knowledge`, source references, and source excerpts - is valid, the chunk is preserved and a bad title may be repaired. This rule directly fixes the v1.03 regression.

7.8. Traceability Rebuild

After sanitation, v1.03.1 ensures IDs and rebuilds source traceability. This prevents traceability entries from pointing to items removed during cleanup.

8. Discussion

8.1. The Main Scientific Result

The main result is not simply that v1.03.1 scores 100% on a preservation rubric. The deeper result is that CKF compilation behaves like a compiler problem. Each compiler pass has a distinct purpose and failure mode: chunking creates stable spans; extraction creates candidate structures; reduction creates a canonical package; promotion improves section allocation; sanitation improves package integrity; traceability rebuilding restores consistency; quality gates expose the result.

8.2. From Format to Compiler

The shallow CKF-0.1-style baseline showed that valid structured output can preserve very little knowledge. v1.0 showed that source-grounded extraction can preserve semantics. v1.01 showed that preserved claims must also be allocated into richer sections. v1.02 showed that quality must be instrumented. v1.03 showed that global sanitation is necessary but dangerous. v1.03.1 showed that sanitation must be field-aware.

This progression supports a core claim: CKF should not be evaluated as a file format alone. It must be evaluated as a compiler pipeline.

8.3. Why v1.03 Failed and v1.03.1 Succeeded

v1.03 failed because it treated field text uniformly. A title, label, tag, name, statement, source excerpt, and ideal answer are not the same kind of object. Requiring a title to be a complete proposition is a category error. v1.03.1 fixed this by assigning field roles and applying stricter checks only to substantive fields.

8.4. Retrieval Readiness as a First-Class Metric

The v1.03 regression demonstrates that a package can appear cleaner while becoming less useful. A CKF package without retrieval chunks is less useful for RAG-style systems. Therefore, retrieval readiness should be a first-class quality dimension. Regression tests should enforce non-zero retrieval chunks and atomic units when the source supports them.

8.5. Traceability Consistency

Removing items after traceability construction can create orphan trace entries. v1.03.1 addresses this by rebuilding traceability after sanitation. This should be considered a required post-sanitization invariant for CKF compilers.

8.6. Remaining Limitations

The main remaining limitation is that `if_then_rules` remains empty in v1.03.1, even though the source contains conditional and decision-like language. Some of this information appears in `decision_rules`, `procedures`, and `playbooks`, so the omission does not break semantic preservation. Nevertheless, v1.04 should promote explicit if-then rules from decision rules when conditions and decisions are available.

A second limitation is that language filtering may still produce false positives on short technical English descriptions. Future filters should require multiple signals before rejecting ASCII-only technical text.

A third limitation is source-title calibration. The v1.03.1 metadata title appears to reflect an internal section heading rather than the document title. Future compilers should prioritize filename, first-page title, or first H1 over later internal headings.

9. Recommendations for v1.04

9.1. Promote If-Then Rules from Decision Rules

When a decision rule has a non-empty `condition` and `decision`, the compiler should generate an if-then rule preserving source references, source excerpts, source basis, and confidence.

9.2. Reduce Language-Filter False Positives

For short ASCII-only technical descriptions, the compiler should require two independent wrong-language signals before removing the item. This would reduce false positives while preserving the gains of the language lock.

9.3. Accept Short Imperative Decisions

Decision fields such as “Use CKF.” or “Use MCP.” can be valid if the condition and reasoning fields are complete. The sanitizer should not reject them merely for being short.

9.4. Improve Source Title Selection

The compiler should prioritize the filename, PDF title, first H1, or first page title over internal headings when setting `metadata.source_title`.

9.5. Expand Regression Tests

The regression suite should add explicit expectations for at least one if-then rule when the source contains conditional language, and for correct source-title inference in the reference document.

10. Threats to Validity

This study has limitations. First, it uses a single reference document. Second, the semantic preservation rubric was manually constructed. Third, compiler outputs depend on LLM behavior, model selection, prompt version, provider, temperature, and deployment configuration. Fourth, structural counts treat all items equally even though some carry more semantic weight than others. Fifth, code audit identifies

plausible mechanisms but does not prove causality without controlled ablations. Sixth, the study evaluates artifacts and compiler code but not downstream agent performance; retrieval and QA benchmarks remain future work.

11. Conclusion

This paper presented a longitudinal evaluation of the CKF compiler through v1.03.1. The project evolved from shallow structured serialization to a preservation-oriented compiler pipeline. The final v1.03.1 deployment preserved 100% of the 20-unit semantic rubric, produced 511 counted CKF elements, restored retrieval readiness with 18 retrieval chunks and 66 atomic units, preserved operational sections with procedures and playbooks, maintained language fidelity, removed truncation artifacts, calibrated metadata, and rebuilt source traceability.

The central lesson is that knowledge compilation quality is multi-dimensional. Semantic preservation is necessary but insufficient. A useful compiler must also allocate knowledge into the right sections, preserve retrieval surfaces, enforce language fidelity, avoid truncation artifacts, calibrate metadata, and maintain traceability consistency after cleanup. CKF v1.03.1 is the first evaluated deployment in this sequence to satisfy these requirements together.

CKF should therefore be understood not merely as a serialization format, but as a compiler methodology for producing AI-consumable intermediate representations of human knowledge. The next research step is downstream evaluation: whether CKF packages improve retrieval precision, answer faithfulness, traceability, token efficiency, and resistance to composition hallucinations relative to raw PDF or unstructured chunks.

Data and Software Availability

The analyzed materials include the reference PDF, CKF JSON, Markdown, and YAML artifacts for compiler v1.03.1, compiler source files, and regression tests. A full reproducibility package should include the source document, CKF artifacts, preservation rubric, structural counting scripts, compiler source code, run manifest, quality outputs, sanitizer audit, and checksums.

Conflict of Interest Statement

The author is affiliated with CKF Research, the organization developing CKF and the CKF compiler. The study reports both improvements and regressions observed during compiler development.

Author Contributions

Paulo Tomazinho designed CKF, supplied the artifacts and compiler source code, conducted the empirical comparisons, and authored the manuscript.

Acknowledgments

The author thanks the communities developing retrieval, knowledge representation, provenance, structured data, and agent interoperability systems.

References

- [1] Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *International Conference on Learning Representations*.
- [2] Bray, T. (2017). The JavaScript Object Notation (JSON) data interchange format. RFC 8259. Internet Engineering Task Force.

- [3] CommonMark. (2024). CommonMark specification, version 0.31.2.
- [4] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., & Larson, J. (2024). From local to global: A Graph RAG approach to query-focused summarization. *arXiv:2404.16130*.
- [5] Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., & Liu, T. (2025). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Computing Surveys*.
- [6] International Organization for Standardization. (2020). ISO 32000-2:2020, Document management — Portable document format — Part 2: PDF 2.0.
- [7] JSON Schema. (2022). JSON Schema draft 2020-12.
- [8] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- [9] Model Context Protocol. (2024). Model Context Protocol specification. Anthropic.
- [10] Moreau, L., Groth, P., Cheney, J., Lebo, T., & Miles, S. (2015). The rationale of PROV. *Journal of Web Semantics*, 35, 235–257.
- [11] Rashkin, H., Nikolaev, V., Lamm, M., Aroyo, L., Collins, M., Das, D., Petrov, S., Tomar, G. S., Turc, I., & Reitter, D. (2023). Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4), 777–840.
- [12] W3C Provenance Working Group. (2013). PROV-O: The PROV ontology. W3C Recommendation.
- [13] YAML Language Development Team. (2021). YAML Ain’t Markup Language (YAML) version 1.2.2.